

Software Requirements Specification (SRS)

for

AuraNet: Autonomous Self-Healing Zero Trust Architecture using eBPF and Federated Learning

Version 1.0.0

Date: 2026-07-03

1. Introduction

This document provides a detailed specification for the AuraNet system, an autonomous, self-healing, zero trust security platform for cloud-native and Kubernetes environments. It outlines the system's purpose, scope, features, and technical requirements for developers, testers, and project stakeholders.

1.1 Purpose

The purpose of AuraNet is to provide an autonomous, self-healing, zero trust security architecture for distributed and containerized workloads. AuraNet combines kernel-level observability and enforcement via eBPF with privacy-preserving, federated machine learning to continuously detect, evaluate, and respond to threats without requiring human intervention. The system aims to close the gap between traditional perimeter-based security and modern, highly dynamic cloud-native environments by enforcing least-privilege access at every request, self-remediating vulnerable code paths through virtual patching, and learning from distributed telemetry without exposing sensitive raw traffic.

1.2 Scope

The AuraNet system will encompass the following key areas:

- **Autonomous Protection:** Continuous, unattended detection, decision-making, and remediation across the protected cluster, with no required human-in-the-loop step for standard operation.
- **Kernel-Level Enforcement:** eBPF-based security sensors and enforcement points operating at the Linux kernel level, including automated virtual patching of vulnerable workloads.
- **Zero Trust Architecture:** Identity-based authentication and authorization for every request, aligned with NIST SP 800-207, including HTTP-layer microsegmentation.
- **AI-Driven Policy Decisions:** Machine learning models embedded in the zero trust policy decision process, with non-blocking shadow-stream inference.
- **Federated Learning & Privacy:** Distributed, privacy-preserving model training across nodes using federated learning, with protections against model poisoning.
- **Extensibility:** Tooling that allows developers to onboard new workloads and applications into the federated learning process.
- **Observability:** A web-based UI dashboard for monitoring system state, threats, and policy decisions.

- **Scalability & Portability:** Horizontal scaling to handle varying load and user counts, packaged via Helm for deployment on any Kubernetes-conformant cluster (managed, kind, Minikube, or bare metal).
- **Secure Transport:** Encrypted communication between all cluster nodes, with an optional post-quantum hybrid mode.
- **Tooling:** A command-line interface (CLI) for installation, configuration, and lifecycle management of the system.

1.3 Definitions, Acronyms, and Abbreviations

- **AI:** Artificial Intelligence
- **CLI:** Command-Line Interface
- **eBPF:** extended Berkeley Packet Filter — a Linux kernel technology that allows sandboxed programs to run in kernel space
- **FIPS 203:** Federal Information Processing Standard specifying the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), the U.S. post-quantum key establishment standard
- **FL:** Federated Learning — a machine learning technique that trains models across decentralized nodes without centralizing raw data
- **L7:** Layer 7 (Application Layer) of the OSI model
- **mTLS:** Mutual Transport Layer Security
- **NIST SP 800-207:** U.S. National Institute of Standards and Technology Special Publication 800-207, "Zero Trust Architecture"
- **PDP / PEP:** Policy Decision Point / Policy Enforcement Point
- **PQC:** Post-Quantum Cryptography
- **UI:** User Interface
- **ZTA:** Zero Trust Architecture

1.4 References

- NIST Special Publication 800-207, "Zero Trust Architecture"
- NIST FIPS 203, "Module-Lattice-Based Key-Encapsulation Mechanism Standard"
- eBPF Foundation Documentation (ebpf.io)
- Helm Documentation (helm.sh)
- Kubernetes Documentation (kubernetes.io)

2. Overall Description

2.1 Product Perspective

AuraNet is a new, self-contained security platform that functions as an autonomous protection layer for Kubernetes-based and containerized environments. It is deployed as a set of cooperating components (kernel-level sensors, a zero trust controller, federated learning nodes, and a management UI) rather than as

an extension of an existing product. Its primary distinction is the combination of kernel-level eBPF enforcement with privacy-preserving federated learning to drive zero trust policy decisions without centralizing sensitive traffic data.

2.2 Product Functions

The major functions of the AuraNet system are:

- **Autonomous Threat Response:** Detects, evaluates, and remediates security events without requiring human approval for standard operation.
- **Kernel-Level Sensing and Enforcement:** Captures security-relevant events at the kernel level via eBPF and applies virtual patches to vulnerable workloads at the same layer.
- **Zero Trust Access Control:** Authenticates and authorizes every request, including HTTP-level microsegmentation between services.
- **AI-Assisted Policy Decisions:** Incorporates machine learning inference into the zero trust controller's decision process via a non-blocking shadow stream.
- **Federated Model Training:** Trains detection models across distributed nodes while preserving the privacy of sensitive network packet data, with safeguards against model poisoning.
- **Workload Onboarding:** Allows developers to register new workloads and applications so that federated learning can begin adapting to them.
- **Observability Dashboard:** Presents system health, threats, and policy activity through a UI dashboard.
- **Horizontal Scaling:** Scales system components to accommodate varying load and user counts.
- **Secure Inter-Node Communication:** Encrypts traffic between all nodes in the cluster, with an optional post-quantum hybrid mode.

2.3 User Characteristics

The system is designed for two primary user groups:

- **Security Operations / Platform Engineers:** Responsible for installing, configuring, and monitoring AuraNet across a cluster. They primarily interact with the system through the UI dashboard and the CLI.
- **Application Developers:** Build and deploy workloads on the protected cluster. AuraNet must remain largely transparent to this group; they interact with the system primarily when onboarding a new workload for federated learning.

2.4 Constraints

- The system's kernel-level components depend on a Linux kernel with eBPF support.
- The system is deployed and managed on Kubernetes-conformant clusters (managed Kubernetes, kind, Minikube, or bare metal).
- The system is packaged and distributed using Helm charts.
- The zero trust controller must comply with the architectural principles of NIST SP 800-207.

- The optional post-quantum encryption mode must be compatible with NIST FIPS 203 (ML-KEM) in a hybrid configuration alongside classical cryptography.
- End-to-end packet latency introduced by the system must remain within a 0-20ms budget for protected workloads.

2.5 Assumptions and Dependencies

- Target clusters run a Linux kernel version that supports the required eBPF features.
- Cluster operators have sufficient privileges to install kernel-level components and Helm charts.
- The security of the system depends on the correctness of the loaded eBPF programs, the integrity of the federated learning process, and the strength of the underlying cryptographic libraries.
- Application developers will use the provided onboarding tooling to register new workloads for federated learning rather than modifying system internals directly.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces (UI)

- **Observability Dashboard:** The system shall provide a web-based UI dashboard that is easy to use and straightforward, giving operators visibility into system state, detected threats, and policy decisions.
- **Command-Line Interface (CLI):** The system shall provide a CLI for installing, uninstalling, configuring, and managing the system across its lifecycle.

3.1.2 Software Interfaces

- **Kernel Interface:** The system must interface with the Linux kernel via verified eBPF programs to capture security events and apply enforcement actions, including virtual patches.
- **Zero Trust Controller Interface:** The system must expose a policy decision/enforcement interface (PDP/PEP) that authenticates and authorizes requests at the HTTP microsegmentation level, consistent with NIST SP 800-207.
- **Federated Learning Interface:** The system must provide an interface allowing distributed nodes to participate in federated model training and allowing developers to register new workloads into that process.
- **Cluster Transport Interface:** The system must interface with the cluster network layer to encrypt traffic between nodes, including support for an optional post-quantum hybrid mode aligned with NIST FIPS 203.
- **Deployment Interface:** The system must be installable via Helm charts on any Kubernetes-conformant cluster.

3.2 Functional Requirements

This section details the functional requirements of the AuraNet system.

FR-1: Autonomous Operation

- **FR-1.1:** The system shall detect, evaluate, and respond to security events autonomously, without requiring human intervention for standard operation.

FR-2: Kernel-Level Security Sensing

- **FR-2.1:** The system shall implement eBPF-based security sensors operating at the kernel level that correctly capture events relevant to security evaluation.
- **FR-2.2:** All eBPF code loaded into the kernel must pass verification and be safe for the kernel to load, in accordance with standard eBPF verifier constraints.

FR-3: Autonomous Self-Healing (Virtual Patching)

- **FR-3.1:** The system shall apply autonomous self-healing at the eBPF kernel level through virtual patches that mitigate identified vulnerabilities without requiring a code deployment or restart of the affected workload.

FR-4: Zero Trust Architecture and Access Control

- **FR-4.1:** The system must implement a zero trust architecture that adheres to the principles defined in NIST SP 800-207.
- **FR-4.2:** The system must apply authentication and authorization at the HTTP microsegmentation level, evaluating every request rather than relying on network-perimeter trust.

FR-5: AI-Assisted Policy Decisions

- **FR-5.1:** The system must integrate machine learning inference into the zero trust controller's decision-making process.
- **FR-5.2:** AI inference used for policy decisions shall run as a non-blocking shadow stream, evaluating traffic in parallel without adding inference latency to the enforcement path.
- **FR-5.3:** The system should account with workload diverse data and non-IID data

FR-6: Federated Learning and Privacy Preservation

- **FR-6.1:** The system must use federated learning to train detection models across distributed nodes while preserving the privacy of sensitive network packet data, avoiding centralization of raw traffic.
- **FR-6.2:** The system should apply engineering safeguards to protect the federated learning process against model poisoning attacks.

FR-7: Workload Extensibility

- **FR-7.1:** The system should allow developers to add new workloads and applications and initiate the federated learning process for them.

FR-8: Observability

- **FR-8.1:** The system shall provide observability into its operation, threats, and decisions through a UI dashboard.

FR-9: Scalability

- **FR-9.1:** The system must scale horizontally to handle varying load and numbers of users.

FR-10: Attack Protection

- **FR-10.1:** The system must protect against common cyberattack techniques, with particular emphasis on Layer 7 (application-layer) attacks.

FR-11: Inter-Node Encryption

- **FR-11.1:** The system must encrypt traffic exchanged between any two nodes in the cluster.

3.3 Non-Functional Requirements

NFR-1: Performance

- Packet-processing latency introduced by the system for workloads and applications built on top of it must remain within a 0-20ms range.

NFR-2: Usability

- The UI dashboard shall be easy to use and straightforward for security operations users.
- The CLI shall provide clear feedback for installation, configuration, and management operations.

NFR-3: Packaging and Portability

- The system shall be packaged using Helm for straightforward installation and portability.
- The system shall be portable and installable on any Kubernetes-conformant cluster, including kind, Minikube, managed Kubernetes, or bare metal.

NFR-4: Developer Transparency

- The system shall remain transparent to application developers and must not interfere with the normal application development workflow.

NFR-5: Adaptability

- The system's machine learning components shall adapt to new workloads as they are onboarded, improving detection accuracy over time without manual retraining steps.

NFR-6: Cryptographic Future-Proofing

- The system shall support an optional post-quantum hybrid encryption mode, combining classical and post-quantum algorithms, in preparation for long-term cryptographic resilience and in adherence to NIST FIPS 203 (ML-KEM).

NFR-7: Tooling

- The system must provide a CLI tool for installing, uninstalling, and managing the system.